

Tecnológico de Monterrey

Campus Monterrey

Situación Problema 1&2

Sophia Alexandra Ramos García A00841358
Marian Guadalupe Willis Lozano A00842035
Regina Elizabeth Flores Sosa A01770709
Isabella Salas Corpus A01571896

Nezih Nieto Gutiérrez

Análisis y diseño de algoritmos avanzados
Gpo 603

2026

Tecnológico de Monterrey

Módulo I. Network Intelligence

Formulación del Modelo

Representamos la red como un grafo no dirigido con pesos, donde cada central es un nodo y cada conexión es una arista con un costo. GÉANT tiene 22 nodos y 36 enlaces, por lo que es un grafo disperso, ya que de las 231 conexiones que podrían existir solo hay 36. Usamos V para el número de nodos, E para los enlaces y N para las centrales del TSP.

En el desafío 01 buscamos conectar todas las centrales con el menor costo y sin formar ciclos, usando un árbol de expansión mínima (MST). En el desafío 02 buscamos una ruta de mantenimiento que comienza en una central, pasa una vez por todas las demás y regresa al inicio con el menor costo, que es el problema del viajante (TSP). Como no todas las centrales están conectadas directamente, usamos el camino más corto entre ellas como distancia.

MST: Kruskal y Prim

Elegimos Kruskal porque los datos vienen como una lista de enlaces y la red tiene pocas conexiones, y además usamos Prim con cola de prioridad para comparar los resultados Kruskal ordena los enlaces de menor a mayor costo y los va seleccionando siempre que no formen ciclos, para revisarlo usa Union-Find, que lo hace casi en O(1), por lo que su complejidad es O(E log E). Prim comienza desde un nodo y en cada paso agrega la conexión más barata que sale del árbol que ya lleva construido, con una complejidad de O(E log V). Kruskal conviene más en redes dispersas como la nuestra y Prim en redes densas.

Ambos algoritmos se basan en la propiedad del corte: si dividimos los nodos en dos grupos, la conexión más barata entre ellos siempre forma parte de algún MST.

TSP: método exacto y método aproximado

Como el TSP es un problema NP-hard, decidimos usar Held-Karp para redes de hasta 22 nodos, ya que encuentra la solución exacta, y vecino más cercano con 2-opt cuando hay más nodos, porque da una solución aproximada en menos tiempo. Así podemos resolver GÉANT completa con Held-Karp, que usa programación dinámica y máscaras de bits para no repetir cálculos. Su complejidad es $O(2^N \cdot N^2)$, mucho menor que revisar las N! rutas posibles por fuerza bruta.

El método aproximado elige siempre la central más cercana y después usa 2-opt para mejorar el recorrido cambiando algunos tramos, con una complejidad de $O(N^2)$ por pasada,

aunque no asegura encontrar la mejor ruta. Held-Karp, en cambio, es correcto por la subestructura óptima: aprovecha los mejores caminos calculados antes para encontrar la ruta de menor costo.

Flujo Máximo: Edmond-Karp

En este problema buscamos determinar la máxima capacidad de transmisión de información que puede enviar un nodo central de origen hacia un nodo central de destino dentro de la red troncal, respetando los límites de capacidad preinstalados en cada enlace de fibra óptica y garantizando la conservación del flujo en los nodos intermedios. Elegimos Edmonds-Karp como el algoritmo principal para esta tarea, comparándolo teóricamente con el algoritmo de Dinic. Edmonds-Karp es una implementación del método de Ford-Fulkerson que utiliza una búsqueda en anchura (BFS) para encontrar sistemáticamente el camino aumentante con el menor número de aristas en el grafo residual. En cada iteración, calcula el cuello de botella (la capacidad residual mínima a lo largo del camino), actualiza los flujos en las aristas directas e inversas y suma dicha capacidad al flujo total acumulado. Su complejidad temporal es de $O(V * E^2)$. En una red dispersa y acotada como GÉANT, esta complejidad garantiza tiempos de ejecución en fracciones de milisegundo con una sobrecarga (overhead) de memoria mínima y un código altamente robusto.

El Edmond-Karp no es muy eficiente para grafos masivos y complicados, es por eso que estos casos la alternativa sería el algoritmo de Dinic, el cual optimiza la búsqueda construyendo un grafo de niveles mediante BFS y saturando múltiples caminos aumentantes en lote mediante una búsqueda en profundidad (DFS). Dinic reduce la complejidad a $O(V^2 * E)$ en el peor caso teórico y a $O(E\sqrt{V})$ en redes unitarias siendo la opción superior para grafos densos o de gran escala, aunque para el tamaño de nuestra red troncal, Edmonds-Karp ofrece la mejor relación entre simplicidad de mantenimiento y eficiencia práctica.

Diagrama de Voronoi

En este problema se particiona el plano bidimensional delimitado por el *bounding box* en regiones poligonales de Voronoi disjuntas, asignando a cada central todos los puntos cuya distancia euclidiana hacia ella sea mínima. Para lograr esto, el código implementa el dual geométrico de la Triangulación de Delaunay mediante el algoritmo incremental de Bowyer-Watson, el cual inserta las centrales punto por punto, elimina los triángulos cuyos circun círculos contienen al nuevo nodo y retriangula la cavidad restante. Los circuncentros de los triángulos resultantes conforman exactamente los vértices de las celdas de Voronoi, los cuales se ordenan angularmente mediante $arctan2$ alrededor de cada central para cerrar su polígono de cobertura.

En la implementación incremental estándar sobre arreglos lineales, la búsqueda de triángulos afectados por cada punto toma un promedio de $O(N\,logN)$. El peor caso, cuando las inserciones generan configuraciones numéricamente degeneradas o cuando cada nuevo punto debe inspeccionar linealmente la totalidad de los triángulos generados hasta el momento, su complejidad sería $O(N^2)$.

Módulo II. Cybersecurity Intelligence

Este módulo busca garantizar la administración de la capacidad de transmisión así como detectar el tráfico malicioso. Se debían implementar algoritmos que pudieran detectar en las transmisiones coincidencias de firmas, palíndromos y subcadenas compartidas.

String matching: Knuth-Morris-Pratt

Para poder identificar posibles firmas de malware desde las transmisiones, se implementó el algoritmo Knuth-Morris-Pratt (KMP). Este algoritmo permite buscar un patrón dentro de una cadena desde los carácteres sin tener que volver a comparar los que ya fueron revisados. Una de las características del algoritmo es que utiliza una función de fallo que guarda la longitud del prefijo más largo que también es sufijo para cada posición del patrón. Cuando se encuentra una diferencia entre caracteres, continúa la búsqueda sin comenzar nuevamente desde el principio ya que la función cambia a dónde debe moverse el puntero del patrón.

Su complejidad es de $O(n\ +\ m)$, donde n es la longitud de la transmisión y m la longitud de la firma. Se eligió el algoritmo de KMP ya que llega a resultar más eficiente cuando las transmisiones son largas o tienen patrones repetidos. Como alternativas, se consideraron tanto Z-Function como Aho-Corasick. Aunque Aho-Corasick permite buscar múltiples firmas simultáneamente y puede ser más eficiente cuando la cantidad de patrones que se debe buscar aumenta, decidimos utilizar KMP como una solución inicial y poder utilizar Aho-Corasick como una alternativa para escalabilidad.

Mirrored Code Analysis: Manacher

Se utilizó el algoritmo de Manacher para encontrar el palíndromo continuo más largo dentro de una cadena. Este se aplica a cada transmisión por separado y se regresan las posiciones iniciales y finales del palíndromo más largo encontrado.

Lo que hace este algoritmo es que primero transforma la cadena al agregar un carácter ‘#’ entre los carácteres y los extremos. Esto permite analizar palíndromos de longitud par o impar de la misma manera para después recorrer la cadena cambiada utilizando un arreglo *P* que guarda cuánto se puede extender cada palíndromo desde su centro. Aprovecha las

posiciones espejo para evitar comparaciones repetidas, manteniendo el centro y un límite derecho del palíndromo que se haya extendido más hacia la derecha.

La complejidad temporal es de $O(n)$, lo que resulta eficiente. Una de las alternativas consideradas era el utilizar la expansión desde cada centro. Sin embargo, esto tendría una complejidad de $O(n^2)$ y el escoger Manacher permite aprovechar la simetría de los palíndromos.

Transmission Similarity Analysis: Programación dinámica

La programación dinámica fue implementada para identificar similitudes entre dos transmisiones. Se implementó para poder encontrar la subcadena continua más larga que las transmisiones tienen en común, lo que quiere decir que debemos buscar los casos donde los caracteres deben aparecer en el mismo orden consecutivamente.

Para esto, se utilizó una matriz donde cada posición guarda la longitud de la subcadena común que termina en los caracteres que se están comparando. Si coinciden, se le suma uno al valor de la posición diagonal anterior. Durante el recorrido se guarda la mayor longitud encontrada y su posición final para calcular el inicio de la subcadena en común.

Se eligió programación dinámica para este proyecto ya que permite reutilizar resultados anteriores en lugar de estar comparando todas las subcadenas posibles, teniendo una complejidad temporal de $O(nm)$, donde n y m son las longitudes de las transmisiones. Como alternativas se consideraron los arreglos de sufijos y el autómata de sufijos que podrían ser eficientes para transmisiones grandes. Sin embargo se terminó escogiendo programación dinámica por tanto su facilidad de implementación como porque permite encontrar la solución exacta usando una matriz sencilla.

Módulo III. Algorithm Laboratory & Scenario Manager

Pruebas (PENDIENTE DE CONFIRMAR)

Probamos los dos métodos de cada problema con 700 grafos aleatorios y comparamos los resultados con fuerza bruta, sin encontrar fallas. También revisamos casos especiales, como redes no conectadas, un solo nodo y conexiones con el mismo costo, los tiempos fueron acordes con la complejidad teórica: en redes dispersas Kruskal fue entre 3 y 4 veces más rápido que Prim, y Held-Karp tardó aproximadamente el doble por cada nodo adicional.

Nodos	Kruskal (ms)	Prim (ms)

2,000	0.73	2.31
100,000	50.7	218.6

Con los 22 nodos de GÉANT, el método aproximado tardó menos de 1 ms, aunque la ruta que obtuvo costó 4.9% más que la óptima, estás pruebas se hicieron en Linux con g++ -O2…..

Edmond Karp vs. Dinic

En esta prueba utiliza un valor de nodos (V) y aristas (E) variables con un flujo máximo definido.

Nodos(V)	Aristas(E)	Flujo Máximo	Edmond Karp	Dinic
10	20	6151	0	0
25	49	1159	0	0
50	169	11966	0	0
100	576	37015	0	0
200	2150	20396	0	0
350	6347	56532	3.9940	0.9960
500	12857	142562	13.1690	0.8350
750	28666	198622	60.8120	1.7170
1000	51234	227708	58.5240	4.9990

Estándares y conclusiones

Organizamos el código con un archivo .h para cada algoritmo y un solo main.cpp, sin librerías externas, y comprobamos que compilara sin advertencias con las mismas opciones del autograder, también agregamos comentarios a las funciones para explicar qué hacen y cuál es su complejidad.

En conclusión, la elección del algoritmo depende de las características de la red. Para el MST, Kruskal resulta más conveniente por la poca cantidad de enlaces, y para el TSP, los 22 nodos de GÉANT permiten usar Held-Karp y obtener la ruta óptima, aunque necesite más recursos.

Para el análisis de Flujo Máximo, la implementación del algoritmo de Edmonds-Karp demostró ser la solución idónea para la topología actual de la red troncal GÉANT. Al tratarse de un grafo disperso de escala acotada (22 nodos y 36 enlaces), este algoritmo permite determinar con precisión la capacidad máxima de transmisión e identificar cuellos de botella en tiempos de ejecución submilimétricos, manteniendo una sobrecarga mínima de memoria y una alta simplicidad de mantenimiento en el código.

Para la generación del diagrama de Voronoi, este algoritmo se seleccionó fundamentalmente por dos razones: por un lado, la propiedad matemática de Voronoi garantiza una división territorial óptima, asegurando que cualquier nueva contratación se asigne de forma automática e inequívoca a la central de servicio más cercana; por otro lado, su derivación a través del dual de la Triangulación de Delaunay mediante el algoritmo de Bowyer-Watson permite obtener los límites poligonales mediante el cálculo de circuncentros, lo que hace posible una implementación *header-only* ligera en C++ nativo sin depender de librerías geométricas externas pesadas.